데이터사이언스

Lecture 09 데이터 프레임 편집

apply()

- 열의 각 값에 함수를 하나씩 적용
- 단순 사칙연산으로 표현하기 어려운 복잡한 조건 로직을 열에 일괄 적용할 때 사용
- axis 옵션:
  - Axis=0 (기본): 열 방향으로 함수 적용
  - axis=1: 행 방향으로 함수 적용 (여러 열 동시 참조)

```
# 기본 구조
df["새열"] = df["기존열"].apply(함수)

def age_group(age):
    if age < 20:   return "10대"
    elif age < 30: return "20대"
    elif age < 40: return "30대"
    else:          return "40대 이상"

df["연령대"] = df["나이"].apply(age_group)
```

lambda

- 간결한 조건부 열 생성
- def 없이 한 줄로 함수 정의 조건이 단순할 때 사용

```
#기본 구조
lambda 매개변수 : 반환값

# 매개변수 1개
f = lambda x: x*2
# 매개변수 2개
f = lambda x, y: x + y

df["할인가"] = df["가격"].apply(lambda x: x * 0.8)

df["등급"] = df["점수"].apply(lambda x: "합격" if x >= 60 else "불합격")
```

np.where & np.select

- 간결한 조건부 열 생성
- np.where: 조건이 2개(True/False)일 때 빠른 벡터 연산

```
#기본 구조
np.where(조건, 참값, 거짓값)

import numpy as np

df["응급여부"] = np.where(df["체온"] > 39.0, "응급", "정상")
```

- np.select: 조건이 3개 이상일 때

```
#기본 구조
np.select(조건리스트, 값리스트, default=기본값)

conditions = [df["점수"]>=90, df["점수"]>=70]
choices = ["A","B"]
df["등급"] = np.select(conditions, choices, default="C")
```

pandas 조건 연산자 | & ~

- |(or): 여러 조건 중 하나라도 True면 포함
- &(and): 모든 조건이 동시에 True여야 함
- ~(not): True/False를 반전

```
# 예시
(df["체온"]>39.0) | (df["수축기혈압"]>180)

(df["나이"]>=65) & (df["응급여부"]==False)

~df["응급여부"]
```

.loc[조건, 열]

- 행 선택과 열 선택을 동시에
- ① [] 단독: 행 필터링만 하기 때문에 .loc 없어도 됨
- ② .loc[조건,열]: 콤마로 열 지정 .loc 필수!
- ③ 조건부 수정: 선택 위치의 값 직접 변경

```
# ① 조건으로 행 필터링 (열 전체)
df[df["이익"] > 0]
df.loc[df["이익"] > 0]          # 동일

# ② 조건 + 특정 열 선택 → .loc 필수!
df.loc[df["이익"] > 0, "상권"]                 # 열 하나
df.loc[df["이익"] > 0, ["상권","매출"]]        # 열 여러 개

# ③ 조건부 값 수정 → .loc 필수!
df.loc[df["응급여부"], "진료과"] = "응급실"
```

퀴즈

◦ 빈 칸을 채우세요.

```
# ① 체온이 39도 초과이거나 수축기혈압이 180 초과인 환자를 필터링하시오.
응급 = (
    (df["체온"] ___ 39.0) ___
    (df["수축기혈압"] ___ 180)
)
df[응급]

# ② 나이 65세 이상이고 응급환자가 아닌 행의 '이름'과 '진료과' 열만 출력하시오.

df.loc[
    (df["나이"] ___ 65) ___ (___df["응급여부"]),
    [___, ___]
]   "이름"    "진료과"
```

groupby()

- 같은 값끼리 묶어서 그룹별로 집계하는 함수

```
#기본 구조
df.groupby("그룹기준열")["집계열"].집계함수()

df.groupby("상권")["이익"].mean()       # 상권별 평균 이익
df.groupby("진료과")["나이"].max()      # 진료과별 최고 나이
df.groupby("상권").size()               # 상권별 행 개수
df.groupby("상권")["이익"].count()      # 상권별 이익의 행의 개수 (결측치 제외)

# agg() – 여러 집계 동시에
df.groupby("상권")["이익"].agg(["mean","max","min","count"])

# 여러 열의 조합을 기준으로 (상권x창업가능)
df.groupby(["상권","창업가능"])["매출"].mean()

# 조건 + groupby
df.loc[df["이익"]>0].groupby("상권").size()
```

퀴즈

- 빈 칸을 채우세요.

```
# ① 상권별 평균 이익을 구하고, 평균 이익이 가장 높은 상권 이름을 출력하시오.

result = df.groupby("상권")["이익"].____()
print(result._____)
             idxmax()

# ② 이익이 0보다 큰 행만 걸러낸 뒤, 상권별 해당 행 수를 구하시오.

df.loc[_____].groupby(_____)._____()
```

결측치

- isnull(): 결측치 확인하기

```
df.isnull().sum()              # 열별 결측치 개수
df.isnull().sum().sum()        # 전체 결측치 수
df[df["나이"].isnull()]        # 결측치 있는 행만 보기
df.info()                      # Non-Null Count 한눈에 확인
```

- fillna(): 결측치 채우기

```
df.fillna(0)
df["나이"].fillna(df["나이"].mean())
df.fillna(method="ffill")  # 앞값
df.fillna(method="bfill")  # 뒷값
```

- dropna(): 결측치 삭제하기

```
df.dropna()            # 결측 행 삭제
df.dropna(axis=1)      # 결측 열 삭제
df.dropna(subset=["나이"])  # 특정 열 기준
```

퀴즈

◦ 빈 칸을 채우세요.

```
# ① 각 열의 결측치 개수를 출력하고, '나이' 열의 결측치를 나이 평균으로 채우시오.

print(df._____().sum())
df["나이"] = df["나이"]._____(
    df["나이"].mean()
)


# ② '이름' 열 기준으로 결측 행을 삭제하고, 나머지 결측치는 0으로 채우시오.

df = df.dropna(subset=[___])
df = df._____(0)
```

isnull
fillna
"이름"

pd.concat()

- 여러 DataFrame을 합칠 때 쓰는 함수
- 월별·연도별 데이터 합산 시 주로 사용

```
import pandas as pd

pd.concat([df1,df2]) #기본값은 axis=0 (위아래로 이어붙임), 인덱스는 그대로 유지
pd.concat([df1,df2],ignore_index=True) #세로로 쌓되, 인덱스를 0부터 새로 매김
pd.concat([df1,df2],axis=1) #가로로 붙이기, 인덱스를 기준으로 옆으로 이어붙임
```

- 아래 데이터로 자유롭게 해보자.

```
df1 = pd.DataFrame({'name': ['Alice', 'Bob'], 'age': [25, 30]}, index=[0, 1])
df2 = pd.DataFrame({'name': ['Charlie', 'David'], 'age': [35, 40]}, index=[0, 1])
```

pd.merge()

- 공통된 키(key)를 기준으로 두 DataFrame을 합치는 함수

```
#inner: 기본, 두 테이블 모두에 있는 행만 반환(교집합)
pd.merge(df1,df2,on="환자ID")
pd.merge(df1,df2,on="환자ID“, how=“inner”) #how 없어도 됨
pd.merge(df1,df2, left_on="ID",right_on="환자ID") #두 DataFrame의 키 컬럼 이름이 다르면 이렇게 따로 지정해야 함

#left: 왼쪽 기준 전체 유지, 오른쪽에 없으면 NaN
pd.merge(df1,df2,on="ID",how="left")

#right: 오른쪽 기준 전체 유지, 왼쪽에 없으면 NaN
pd.merge(df1,df2,on="ID",how="right")

#outer: 양쪽 모두 포함, 없으면 NaN (합집합)
pd.merge(df1,df2,on="ID",how="outer")
```

pd.merge()

- 아래 데이터를 만들어보자.

```
# 환자 기본정보
df1 = pd.DataFrame({'환자ID': ['P001', 'P002', 'P003', 'P004'], '나이': [45, 32, 58, 27], '성별': ['M', 'F', 'M', 'F']})

# 검사 결과
df2 = pd.DataFrame({'ID': ['P001', 'P002', 'P003', 'P005'], '혈압': [120, 135, 140, 118], '혈당': [95, 110, 130, 88]})
```

- df1과 df2를 합쳐서 환자 기본정보와 검사 결과를 한 번에 볼 수 있게 만들어보자. 단, 두 데이터에 모두 존재하는 환자만 포함해보자.
- 환자ID와 ID 컬럼이 둘 다 남아있는데, 중복되는 컬럼을 제거하려면?
- 검사 받지 않은 환자도 누락되지 않도록, df1의 모든 환자를 유지하면서 검사 결과를 붙여보자.

퀴즈

◦ 빈 칸을 채우세요.

```
# ① df_서울과 df_부산을 위아래로 연결하고 인덱스를 0부터 재정렬하시오.

result = pd.concat([df_서울, df_부산], ignore_index=True)


# ② df_환자(환자ID,이름)와 df_진료(환자ID,진료과,최종진료비)를 환자ID 기준 왼쪽 병합하시오.

result = pd.merge(df_환자, df_진료, on='환자ID', how='left')
```

pivot_table()

- 행·열·값을 재구조화하는 강력한 도구
- 긴 형태(long format)의 데이터를 넓은 형태(wide format)로 바꾸면서 집계까지 한 번에 해주는 함수

```
pd.pivot_table(df,
               values="집계할 열",       # 어떤 숫자를 집계할지
               index="행으로 쓸 열",     # 행 인덱스
               columns="열로 쓸 열",    # 열 헤더
               aggfunc="집계방식",       # mean(기본), sum, count, max
               fill_value=0)            # NaN 대체값

# 예시: 상권×창업가능여부 이익 합계
pd.pivot_table(df,values="이익",index="상권",
               columns="창업가능",aggfunc="sum",fill_value=0)

# margins=True — 합계 행/열 추가
pd.pivot_table(df,values="매출",index="상권",
               aggfunc="sum",margins=True,margins_name="합계")
```

pivot_table()

- 아래 데이터를 만들어보자.

```
df = pd.DataFrame({
   '상권': ['강남', '강남', '강남', '홍대', '홍대', '홍대', '이태원', '이태원'],
   '업종': ['카페', '식당', '카페', '카페', '식당', '식당', '카페', '식당'],
   '창업가능': ['Y', 'Y', 'N', 'Y', 'Y', 'N', 'Y', 'N'],
   '매출': [100, 200, 120, 150, 180, 90, 130, 160],
   '이익': [30, 60, 40, 45, 50, 20, 35, 45]})
```

- 상권별 평균 매출은?
- 상권 × 업종별 매출 평균은?
- 상권 × 업종별 매출 합계는?
- 상권별 매출 평균, 매출 합계, 건수는?
- 상권 × 창업가능여부별 이익 합계는?
- 상권별 매출 합계 + 전체 총합도 같이 출력해보자.

.str 접근자

- 문자열 열 처리
- 파이썬 문자열 메소드를 열 전체에 한 번에 적용

```
df["이름"].str.upper()                # 대문자
df["이름"].str.lower()                # 소문자
df["이름"].str.strip()                # 앞뒤 공백 제거

df["주소"].str.contains("서울")     # True/False 반환
df["코드"].str.startswith("A")
df["코드"].str.endswith("01")

df["전화"].str.replace("-","")      # - 제거
df["날짜"].str.split("-")           # 리스트로 분리
df["날짜"].str.split("-").str[0]    # 연도만 추출
df["코드"].str[0:3]                 # 슬라이싱
df["이름"].str.len()                # 문자 길이

# 필터링과 결합
df[df["주소"].str.contains("서울")]    # 서울 거주자만
```

퀴즈

◦ 빈 칸을 채우세요.

```
# ① '주소' 열에서 '서울'이 포함된 행만 필터링하시오.

서울거주 = df[
    df["주소"].str.__contains__("서울")]

#대소문자 무시? NaN있을 때? 정규식(regex)?

# ② '날짜' 열이 '2024-03-15' 형식일 때, 연도만 추출해서 '연도' 열로 저장하시오.

# 방법 1: split 활용
df["연도"] = df["날짜"].str.__split__("-").str[__0__]

# 방법 2: 슬라이싱 활용
df["연도"] = df["날짜"].str[__0__:__4__]
```

dtype

- 데이터 타입

pandas 타입	의미	예시
int	정수	나이, 개수, ID
float	실수	체중, 혈압, 검사수치
object	문자열 (또는 혼합)	이름, 주소, 범주
bool	True/False	합격/불합격, 환자/정상
datetime	날짜·시간	생년월일, 진료일
category	범주형	성별, 등급, 진료과

dtype

- 타입 확인하기

```
# 전체 컬럼의 타입 확인
df.dtypes

# 특정 컬럼만
df["나이"].dtype

# 더 자세히 (메모리, 결측치 포함)
df.info()
```

- 기본 타입 변환

```
df["나이"] = df["나이"].astype(int)       # 정수로
df["가격"] = df["가격"].astype(float)     # 실수로
df["성별"] = df["성별"].astype("category") # 범주형으로
df["합격"] = df["합격"].astype(bool)      # True/False로
```

dtype

- 다음 코드를 실행해보자.

```
df = pd.DataFrame({"나이": ["25", "30", "N/A", "45"]})
df["나이"].astype(int)
```

- 어떤 에러가 발생하는가? ValueError
- 해결책:

```
df["나이"] = pd.to_numeric(df["나이"], errors="coerce")
```

- 날짜 변환:

```
pd.to_datetime(df["날짜"], errors="coerce")
pd.to_datetime(df["날짜"], format="%Y-%m-%d")          #2025-04-15
pd.to_datetime(df["날짜"], format="%Y년 %m월 %d일")    # 2024년 3월 15일
pd.to_datetime(df["날짜"], format="%Y.%m.%d")         # 2024.03.15
pd.to_datetime(df["날짜"], format="%Y%m%d")           # 20240315
```

9주차 과제

여러분은 종합 쇼핑몰 "SmartShop"의 신입 데이터 분석가입니다. 회사는 전자기기·패션잡화·뷰티·스포츠·생활용품을 판매하는 온라인 쇼핑몰이며,고객 등급제(VIP/GOLD/SILVER/BRONZE)로 충성도를 관리하고 있습니다. 아래 6개 데이터는 각각 다른 부서(CRM팀, 상품팀, 주문팀, CS팀)에서 관리되어 컬럼명과 형식이 제각각이라는 점에 유의하세요.

1. customers.csv – 고객 기본정보
2. products.csv – 상품 정보
3. orders_2024XX.csv – 월별 주문 데이터
4. reviews.csv – 상품 리뷰

Eclass에 9주차 과제를 푸세요.